Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота № 6

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

« Побудування програмної системи з множини об'єктів, керованих повідомленнями »

Виконав:

студент групи ІМ-53
Брагар Артем Сергійович
Номер в списку - 2

Перевірив:

Рекечинський Д. О.

Київ 2026

Завдання

1. Створити у середовищі розробки проект з ім’ям Lab6.

2. Написати вихідні тексти усіх програм-компонентів згідно варіанту завдання.

3. Скомпілювати вихідні тексти і отримати виконувані файли програм.

4. Перевірити роботу програм. Налагодити взаємодію програм.

5. Проаналізувати та прокоментувати результати та вихідні тексти програм.

6. Оформити звіт

Завдання згідно варіанту:

1. 1. Користувач вводить значення параметрів n, Min, Max у діалоговому вікні. 2. Програма викликає програми Object2, Object3 і забезпечує обмін повідомленнями для передавання та отримання інформації.
2. 1. Створює вектор n дробових (double) чисел у діапазоні Min – Max. 2. Показує числові значення у декількох стовпчиках та рядках у власному головному вікні 3. Записує дані в буфер обміну у текстовому форматі
3. 1. Зчитує дані з буферу обміну 2. Виконує сортування масиву чисел і відображає його у декількох стовпчиках та рядках у власному головному вікні

Файли Лабораторної
Файли розширення(.cs)

Lab6/ChildProgram

```
namespace Lab6;

using System.Diagnostics;

public class ChildProgram
```

```
                        {
        private readonly string programName;
          private Process? process = null;
          public ChildProgram(string name)
                        {
                  programName = name;
                        }
                public void Start()
                        {
    if(process != null && !process.HasExited){return;}
              Process[] runningProcesses =
     Process.GetProcessesByName(programName);
     for(int i = 1; i < runningProcesses.Length; i++)
                        {
                runningProcesses[i].Dispose();
                        }
              if(runningProcesses.Length > 0)
                        {
                process = runningProcesses[0];
                        return;
                        }
  string exePath = Path.Combine(AppContext.BaseDirectory,
              programName + ".exe");
            process = Process.Start(exePath);
    if(process != null){process.WaitForInputIdle();};
                        }
                public void Close()
                        {
```

```
            if(process != null && !process.HasExited)
                                {
                              process.Kill();
                            process.Dispose();
                              process = null;
                                }
                             }
                              }
```

Lab6/Form1

```
                        namespace Lab6;
                     using System.Globalization;


                public partial class Form1 : Form
                                {
        private const string Object2PipeName = "Lab6.Object2";
        private const string Object3PipeName = "Lab6.Object3";
                 private const int Lab6Height = 100;
                    private ChildProgram object2;
                    private ChildProgram object3;
                            public Form1()
                                {
                           InitializeComponent();
          Rectangle area = Screen.FromControl(this).WorkingArea;
                 StartPosition = FormStartPosition.Manual;
          Bounds = new Rectangle(area.Left, area.Top, area.Width,
                           Lab6Height);
```

```
                CreateMenu();
            object2 = new ChildProgram("Object2");
            object3 = new ChildProgram("Object3");
                this.FormClosed += Form_Closed;
                }
                private void CreateMenu()
                {
            MenuStrip menuStrip = new MenuStrip();
        ToolStripMenuItem workMenuItem = new ToolStripMenuItem("Робота");
            ToolStripMenuItem runMenuItem = new
            ToolStripMenuItem("Запустити");
                Controls.Add(menuStrip);
                MainMenuStrip = menuStrip;
            menuStrip.Items.Add(workMenuItem);
        workMenuItem.DropDownItems.Add(runMenuItem);
            runMenuItem.Click += RunMenuItem_Click;
                }
        private void Form_Closed(object? sender, EventArgs e)
                {
                object2.Close();
                object3.Close();
                }
    private async void RunMenuItem_Click(object? sender, EventArgs e)
                {
        using (ParametersDialog parametersDialog = new ParametersDialog())
                    {
            if(parametersDialog.ShowDialog() == DialogResult.Yes)
```

```
                    {
                     try
                      {
string message = BuildMessage(parametersDialog);
 bool object2Ok = await ExchangeAsync(object2,
 Object2PipeName, message);
                if (!object2Ok)
                      {
   MessageBox.Show("Object2 повернув помилку");
                     return;
                      }
 bool object3Ok = await ExchangeAsync(object3,
 Object3PipeName, "READ");
                if (!object3Ok)
                      {
   MessageBox.Show("Object3 повернув помилку");
                     return;
                      }
                    }
           catch (TimeoutException)
                    {
MessageBox.Show("Одна з програм не відповіла");
                    }
             catch (IOException)
                    {
MessageBox.Show("Зв'язок з програмою перервано");
                    }
                  }
```

```
                }

            }

    private string BuildMessage(ParametersDialog parametersDialog)

            {

        string countValue = parametersDialog.CountValue.ToString();

                string minValue =
parametersDialog.MinValue.ToString(CultureInfo.InvariantCulture);

                string maxValue =
parametersDialog.MaxValue.ToString(CultureInfo.InvariantCulture);

        return string.Join(";", countValue, minValue, maxValue);

            }

  private async Task<bool> ExchangeAsync(ChildProgram childProgram,
              string channelName, string message)

            {

                childProgram.Start();

    string? answer = await PipeSender.AsyncSend(channelName, message);

                return answer == "OK";

            }

        }
```

Lab6/PipeSender

```
            namespace Lab6;

            using System.IO.Pipes;


        internal static class PipeSender

            {

public static async Task<string?> AsyncSend(string channelName, string
                channelMessage)
```

```
                {
            using NamedPipeClientStream stream = new
NamedPipeClientStream(".", channelName, PipeDirection.InOut);
            await stream.ConnectAsync(5000);
        using StreamReader reader = new StreamReader(stream);
        using StreamWriter writer = new StreamWriter(stream);
            writer.AutoFlush = true;
        await writer.WriteLineAsync(channelMessage);
            return await reader.ReadLineAsync();
                }
            }
```

Object2/Form1

```
            namespace Object2;
              using Shared;
            using System.IO.Pipes;
          using System.Globalization;
      using System.Runtime.InteropServices;


        public partial class Form1 : Form
                {
    private const string PipeName = "Lab6.Object2";
          private const int Lab6Height = 100;
    private VectorTable vectorTable = new VectorTable();
            public Form1()
                {
            InitializeComponent();
```

```
                    Controls.Add(vectorTable);
        Rectangle area = Screen.FromControl(this).WorkingArea;
                StartPosition = FormStartPosition.Manual;
        Bounds = new Rectangle(area.Left, area.Top + Lab6Height,
            area.Width / 2, area.Height - Lab6Height);
                    this.Load += Form_Load;
                                }
    private async void Form_Load(object? sender, EventArgs e)
                                {
                            while (true)
                                  {
                                   try
                                    {
                    using NamedPipeServerStream stream = new
    NamedPipeServerStream(PipeName, PipeDirection.InOut);
                     await stream.WaitForConnectionAsync();
                using StreamReader reader = new StreamReader(stream);
                using StreamWriter writer = new StreamWriter(stream);
                            writer.AutoFlush = true;
                    string? line = await reader.ReadLineAsync();
                            string answer = "ERROR";
                     if(line != null && ProcessMessage(line))
                                        {
                                   answer = "OK";
                                        }
                      await writer.WriteLineAsync(answer);
                                    }
                             catch (IOException)
```

```
                                    {
                                await Task.Delay(100);
                                    }


                                }
                            }
            private bool ProcessMessage(string message)
                                {
                    string[] parts = message.Split(';');
                    if(parts.Length != 3){return false;}
    if(!int.TryParse(parts[0], out int count) || count <= 0){return
                                false;}
            if(!double.TryParse(parts[1], NumberStyles.Float,
CultureInfo.InvariantCulture, out double min)){return false;}
            if(!double.TryParse(parts[2], NumberStyles.Float,
CultureInfo.InvariantCulture, out double max)){return false;}
   double[] generatedNumbers = VectorGenerator.Generate(count, min,
                                max);
             Text = $"n = {count}; min = {min}; max = {max}";
                vectorTable.ShowVector(generatedNumbers);
                return CopyToClipboard(generatedNumbers);
                                }
            private bool CopyToClipboard(double[] items)
                                {
                string text = VectorFormat.ToText(items);
                                 try
                                  {
                            Clipboard.SetText(text);
```

```
                return true;
            }
        catch (ExternalException)
            {
MessageBox.Show("Інша пргорама не дає можливості змінити вміст буферу обміну");
            return false;
            }
        }
    }
```

Object2/VectorGenerator

```
namespace Object2;
internal static class VectorGenerator
    {
public static double[] Generate(int amount, double min, double max)
        {
        double[] items = new double[amount];
        for(int i = 0; i < items.Length; i++)
            {
            double r = Random.Shared.NextDouble();
            double value = min + r * (max - min);
            items[i] = value;
            }
        return items;
        }
    }
```

Object3/Form1

```
namespace Object3;

using Shared;

using System.IO.Pipes;

using System.Runtime.InteropServices;

public partial class Form1 : Form

{

private const string PipeName = "Lab6.Object3";

private const int Lab6Height = 100;

private VectorTable vectorTable = new VectorTable();

public Form1()

{

InitializeComponent();

Controls.Add(vectorTable);

Rectangle area = Screen.FromControl(this).WorkingArea;

StartPosition = FormStartPosition.Manual;

Bounds = new Rectangle(area.Left + area.Width / 2, area.Top +
Lab6Height, area.Width / 2, area.Height - Lab6Height);

Text = "Object3";

this.Load += Form_Load;

}

private async void Form_Load(object? sender, EventArgs e)

{

while (true)

{

try
```

```
                                    {
                using NamedPipeServerStream stream = new
        NamedPipeServerStream(PipeName, PipeDirection.InOut);
                 await stream.WaitForConnectionAsync();
            using StreamReader reader = new StreamReader(stream);
            using StreamWriter writer = new StreamWriter(stream);
                        writer.AutoFlush = true;
                string? line = await reader.ReadLineAsync();
                        string answer = "ERROR";
                    if(line == "READ" && ReadAndSort())
                                    {
                                answer = "OK";
                                    }
                  await writer.WriteLineAsync(answer);
                                    }
                           catch (IOException)
                                    {
                            await Task.Delay(100);
                                    }


                                  }
                                }
                        private bool ReadAndSort()
                                    {
                               string text = "";
                                try
                                  {
```

```
                text = Clipboard.GetText();
            } catch (ExternalException)
                {
                return false;
                }
        if(!VectorFormat.TryParse(text, out double[] items)){return
                false;}
            Array.Sort(items);
        vectorTable.ShowVector(items);
            return true;
            }
            }
```

Shared/VectorFormat

```
            namespace Shared;
        using System.Globalization;
    public static class VectorFormat
            {
        public static string ToText(double[] items)
            {
        string[] texts = items.Select(x =>
    x.ToString(CultureInfo.InvariantCulture)).ToArray();
        return string.Join(';', texts);
            }
    public static bool TryParse(string text, out double[] items)
            {
        items = Array.Empty<double>();
```

```
            if(string.IsNullOrEmpty(text)){return false;}

                  string[] parts = text.Split(';');

                double[] values = new double[parts.Length];

                   for(int i =0; i < values.Length; i++)

                                        {

                if(!double.TryParse(parts[i], NumberStyles.Float,
            CultureInfo.InvariantCulture, out values[i]))

                                          {

                                    return false;

                                        }

                                      }

                                items = values;

                                 return true;

                                    }

                                   }
```

Shared/VectorTable

```
                            namespace Shared;

                    public class VectorTable : DataGridView

                                        {

                       private const int ColumnsAmount = 5;

                                public VectorTable()

                                          {

                                    ReadOnly = true;

                                AllowUserToAddRows = false;

                                  Dock = DockStyle.Fill;

                AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
```

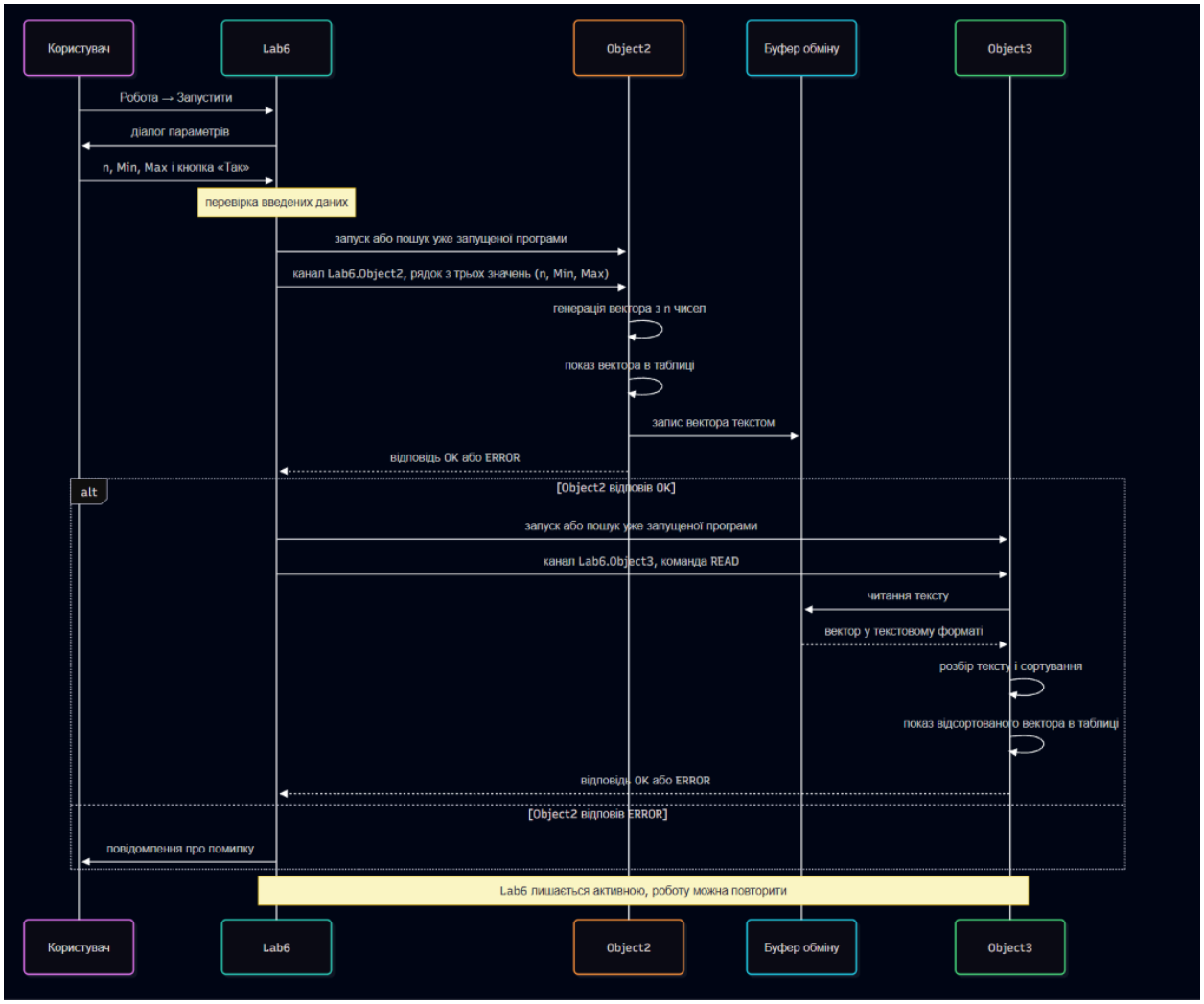
```
                }
        public void ShowVector(double[] items)
                {
                    Rows.Clear();
                   Columns.Clear();
        int rowsAmount = (items.Length + ColumnsAmount -1) /
                    ColumnsAmount;
           for(int i = 0; i < ColumnsAmount; i++)
                    {
        Columns.Add($"{i+1}-Item", (i + 1).ToString());
                    }
         for (int row = 0; row < rowsAmount; row++)
                    {
          object[] cells = new object[ColumnsAmount];
      for(int column = 0; column < ColumnsAmount; column++)
                      {
              int index = row * ColumnsAmount + column;
                    if (index < items.Length)
                      {
             cells[column] = items[index].ToString("F2");
                      }
                     else
                      {
                   cells[column] = "";
                      }
                    }
                  Rows.Add(cells);
```

```
                }

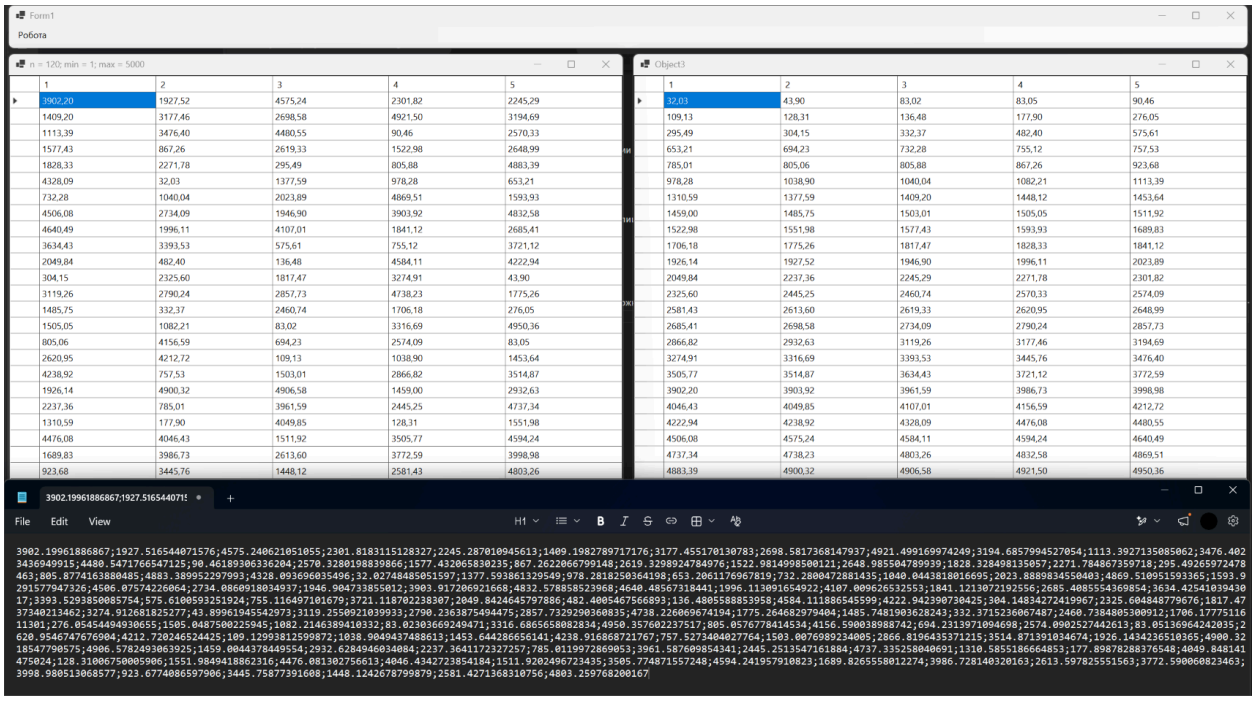
            }

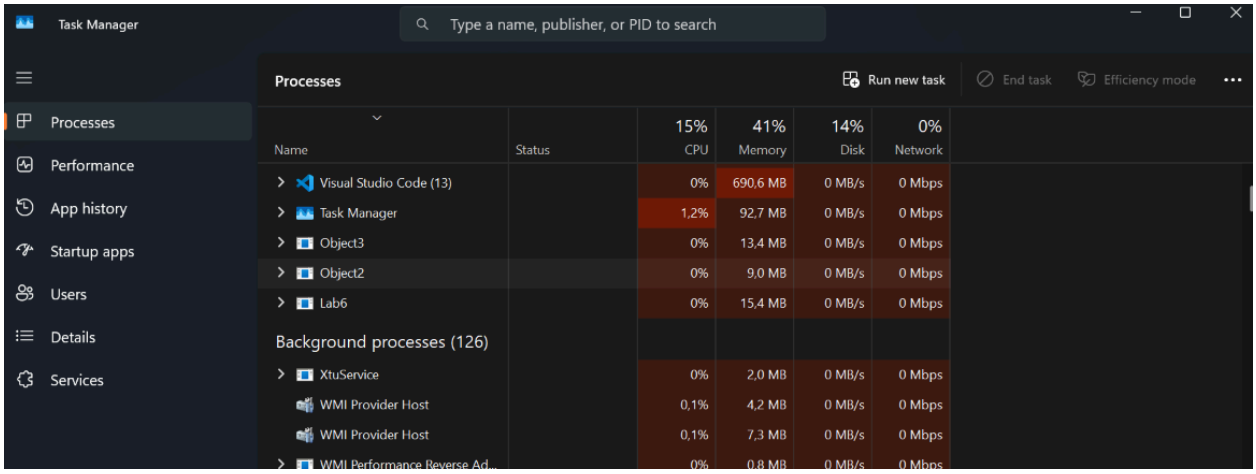
        }
```

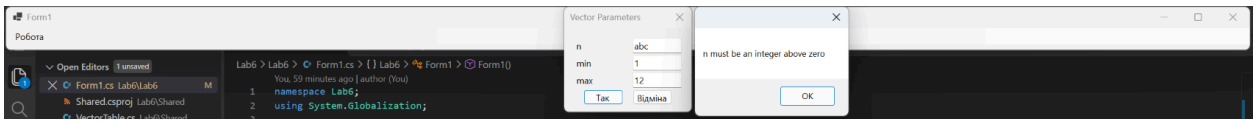
Діаграма


Користувач
Lab6
Object2
Буфер обміну
Object3
Робота → Запустити
діалог параметрів
n, Min, Max і кнопка «Так»
перевірка введених даних
запуск або пошук уже запущеної програми
канал Lab6.Object2, рядок з трьох значень (n, Min, Max)
генерація вектора з n чисел
показ вектора в таблиці
запис вектора текстом
відповідь OK або ERROR
alt
[Object2 відповів OK]
запуск або пошук уже запущеної програми
канал Lab6.Object3, команда READ
читання тексту
вектор у текстовому форматі
розбір тексту і сортування
показ відсортованого вектора в таблиці
відповідь OK або ERROR
[Object2 відповів ERROR]
повідомлення про помилку
Lab6 лишається активною, роботу можна повторити
Користувач
Lab6
Object2
Буфер обміну
Object3

Скріншоти


Form1
Робота
n = 120; min = 1; max = 5000
Object3
3902.19961886867;1927.516544071!
File Edit View


Task Manager
Type a name, publisher, or PID to search
Processes
Run new task
End task
Efficiency mode
Processes
Performance
App history
Startup apps
Users
Details
Services
Name
Status
15% CPU
41% Memory
14% Disk
0% Network
Visual Studio Code (13) 0% 690,6 MB 0 MB/s 0 Mbps
Task Manager 1,2% 92,7 MB 0 MB/s 0 Mbps
Object3 0% 13,4 MB 0 MB/s 0 Mbps
Object2 0% 9,0 MB 0 MB/s 0 Mbps
Lab6 0% 15,4 MB 0 MB/s 0 Mbps
Background processes (126)
XtuService 0% 2,0 MB 0 MB/s 0 Mbps
WMI Provider Host 0,1% 4,2 MB 0 MB/s 0 Mbps
WMI Provider Host 0,1% 7,3 MB 0 MB/s 0 Mbps


Form1
Робота
Vector Parameters
n must be an integer above zero
OK

Висновки:

У ході роботи я навчився створювати систему з трьох незалежних програм, якими керує головна програма Lab6, та організовувати обмін даними між процесами. Я використав іменовані канали для передавання команд і відповідей, а буфер обміну для передавання масиву чисел у текстовому форматі. Я зрозумів, чому формат даних, що передаються між програмами, потрібно фіксувати незалежно від мови операційної системи, і навчився керувати запуском та завершенням інших процесів, зокрема коли вони вже були запущені. Також я на практиці побачив, навіщо виносити спільний код у окрему бібліотеку замість його копіювання між проєктами.